

스택 프로그램 설명 및 동작 결과 정리

이 문서는 각 스택 관련 프로그램의 코드 전문, 동작 설명, 실행 순서 및 실행 결과를 포함한 상세한 해설을 제공합니다.

🔗 프로그램 4.1 - 정수 배열 스택 프로그램

🔍 코드 전문

```
#define MAX_STACK_SIZE 100
typedef int element;
element stack[MAX_STACK_SIZE];
int top = -1;

int is_empty(){ return (top == -1); }
int is_full(){ return (top == (MAX_STACK_SIZE - 1)); }

void push(element item){
    if (is_full()){
        fprintf(stderr, "스택 포화 에러\n");
    } else stack[++top] = item;
}

element pop(){
    if (is_empty()){
        fprintf(stderr, "스택 공백 에러\n");
    } else return stack[top--];
}

element peek(){
    if (is_empty()){
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    } else return stack[top--];
}

int main(){
    push(1);
    push(2);
    push(3);
    printf("%d\n", pop());
    printf("%d\n", pop());
    printf("%d\n", pop());
    return 0;
}
```

🗨 설명

- 배열 기반 스택을 이용한 기본적인 `push`, `pop`, `peek` 연산 구현.
- `is_empty` 및 `is_full`로 상태 체크.
- 배열 `stack[]`의 `top` 값을 이용하여 데이터를 삽입/삭제함.

⚙ 동작 순서

1. `push(1)`, `push(2)`, `push(3)` 으로 스택에 1, 2, 3 저장.
2. `pop()` 3 → 출력
3. `pop()` 2 → 출력
4. `pop()` 1 → 출력

☑ 실행 결과

```
3
2
1
```

📌 프로그램 4.2 - 구조체 배열 스택 프로그램

🔍 코드 전문

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_STACK_SIZE 100
#define MAX_STRING 100

typedef struct {
    int student_no;
    char name[MAX_STRING], address[MAX_STRING];
} element;

element stack[MAX_STACK_SIZE];
int top = -1;

int is_empty(){ return (top == -1); }
int is_full(){ return (top == (MAX_STACK_SIZE - 1)); }

void push(element item){
    if (is_full()){
        fprintf(stderr, "스택 포화 에러\n");
    } else stack[++top] = item;
}

element pop(){
    if (is_empty()){
        fprintf(stderr, "스택 공백 에러\n");
    } else return stack[top--];
}
```

```

}

element peek(){
    if (is_empty()){
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    } else return stack[top--];
}

int main(){
    element ie = {20190001, "Hong", "Seoul"};
    element oe;
    push(ie);
    oe = pop();

    printf("학번: %d\n", oe.student_no);
    printf("이름: %s\n", oe.name);
    printf("주소: %s\n", oe.address);
    return 0;
}

```

프로그램 설명

이 프로그램은 구조체 `element`를 사용하여 학생 정보를 저장하는 배열 기반 스택을 구현합니다. 스택의 기본 연산(push, pop, peek)은 구조체 단위로 작동합니다.

구조체 필드

- `student_no`: 학번 (정수)
- `name`: 이름 (문자열)
- `address`: 주소 (문자열)

핵심 함수

- `push()` : 구조체 데이터를 스택에 삽입
- `pop()` : 가장 위에 있는 구조체 데이터를 꺼냄
- `peek()` : 현재는 top 요소를 반환하고 제거

동작 순서

1. 구조체 `ie`를 선언하고 값 할당 (학번, 이름, 주소)
2. `push(ie)`로 스택에 저장
3. `pop()`으로 꺼내 `oe`에 저장
4. `oe`의 각 필드를 출력

실행 결과

학번: 20190001
이름: Hong
주소: Seoul

프로그램 4.3 배열 기반 스택 프로그램 (구조체 사용)

코드 전문

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_STACK_SIZE 100

typedef int element;
typedef struct {
    element data[MAX_STACK_SIZE];
    int top;
} StackType;

void init_stack(StackType *s){ s->top = -1; }
int is_empty(StackType *s){ return (s->top == -1); }
int is_full(StackType *s){ return (s->top == (MAX_STACK_SIZE - 1)); }

void push(StackType *s, element item){
    if (is_full(s)){
        fprintf(stderr, "스택 포화 에러\n");
        return;
    } else s->data[++(s->top)] = item;
}

element pop(StackType *s){
    if (is_empty(s)){
        fprintf(stderr, "스택 공백 에러\n");
    } else return s->data[(s->top)--];
}

element peek(StackType *s){
    if (is_empty(s)){
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    } else return s->data[s->top];
}

int main(){
    StackType s;
    init_stack(&s);
    push(&s, 1);
    push(&s, 2);
```

```

    push(&s, 3);
    printf("%d\n", pop(&s));
    printf("%d\n", pop(&s));
    printf("%d\n", pop(&s));
    return 0;
}

```

설명

이 프로그램은 **배열 기반 스택**을 구조체로 구현한 예제입니다.

StackType 구조체는 정수 배열 **data[]**와 현재 스택의 최상단 인덱스를 저장하는 **top** 필드를 포함합니다.

- **init_stack**: 스택을 초기화합니다.
- **push**: 스택에 값을 삽입합니다. 포화 상태일 경우 에러 출력.
- **pop**: 스택에서 값을 꺼냅니다. 공백 상태일 경우 에러 출력.
- **peek**: 스택의 최상단 값을 확인합니다. (삭제하지 않음)

실행 순서

1. **init_stack(&s)** → 스택 초기화
2. **push(&s, 1)** → 스택: [1]
3. **push(&s, 2)** → 스택: [1, 2]
4. **push(&s, 3)** → 스택: [1, 2, 3]
5. **pop(&s)** → 출력: 3
6. **pop(&s)** → 출력: 2
7. **pop(&s)** → 출력: 1

실행 결과

```

3
2
1

```

프로그램 4.4 - 포인터 기반 동적 스택 프로그램

코드 전문

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_STACK_SIZE 100

typedef int element;

```

```
typedef struct {
    element data[MAX_STACK_SIZE];
    int top;
} StackType;

void init_stack(StackType *s){ s->top = -1; }
int is_empty(StackType *s){ return (s->top == -1); }
int is_full(StackType *s){ return (s->top == (MAX_STACK_SIZE - 1)); }

void push(StackType *s, element item){
    if (is_full(s)){
        fprintf(stderr, "스택 포화 에러\n");
        return;
    } else s->data[++(s->top)] = item;
}

element pop(StackType *s){
    if (is_empty(s)){
        fprintf(stderr, "스택 공백 에러\n");
    } else return s->data[(s->top)--];
}

element peek(StackType *s){
    if (is_empty(s)){
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    } else return s->data[s->top];
}

int main(){
    StackType *s;
    s = (StackType *)malloc(sizeof(StackType));
    init_stack(s);
    push(s, 1);
    push(s, 2);
    push(s, 3);
    printf("%d\n", pop(s));
    printf("%d\n", pop(s));
    printf("%d\n", pop(s));
    free(s);
    return 0;
}
```

설명

이 프로그램은 포인터를 이용해 동적으로 스택 구조체를 생성하는 예제입니다.

정적 선언이 아닌 `malloc`을 통해 스택 구조체 메모리를 생성하고, 사용 후 `free`로 해제합니다.

- `init_stack`: `top`을 -1로 초기화
- `push`: 배열이 가득 찼는지 검사 후 삽입
- `pop`: 비어있으면 에러 출력, 아니면 `top` 값 제거

- `peek` : 가장 위의 값 확인 (삭제 없이)

⚙ 실행 순서

1. `malloc`으로 스택 구조체 동적 할당
2. `init_stack(s)` → 스택 초기화
3. `push(s, 1)` → 스택: [1]
4. `push(s, 2)` → 스택: [1, 2]
5. `push(s, 3)` → 스택: [1, 2, 3]
6. `pop(s)` → 출력: 3
7. `pop(s)` → 출력: 2
8. `pop(s)` → 출력: 1
9. `free(s)` → 메모리 해제

☑ 실행 결과

```
3
2
1
```

📌 프로그램 4.5 - 동적 크기 증가 스택

🔍 코드 전문

```
#include <stdio.h>
#include <stdlib.h>

typedef int element;
typedef struct {
    element *data;
    int capacity;
    int top;
} StackType;

void init_stack(StackType *s){
    s->top = -1;
    s->capacity = 1;
    s->data = (element *)malloc(s->capacity * sizeof(element));
}

int is_empty(StackType *s){ return (s->top == -1); }
int is_full(StackType *s){ return (s->top == (s->capacity - 1)); }

void push(StackType *s, element item){
```

```

    if (is_full(s)){
        s->capacity *= 2;
        s->data = (element *)realloc(s->data, s->capacity * sizeof(element));
    }
    s->data[++(s->top)] = item;
}

element pop(StackType *s){
    if (is_empty(s)){
        fprintf(stderr, "스택 공백 에러\n");
    } else return s->data[(s->top)--];
}

int main(){
    StackType s;
    init_stack(&s);
    push(&s, 1);
    push(&s, 2);
    push(&s, 3);
    printf("%d\n", pop(&s));
    printf("%d\n", pop(&s));
    printf("%d\n", pop(&s));
    free(s.data);
    return 0;
}

```

설명

이 프로그램은 스택의 크기를 동적으로 확장하는 배열 기반 스택 구현입니다.
스택이 가득 찼을 때 `realloc()`을 이용해 용량을 2배로 늘려 자동 확장합니다.

- `init_stack`: 초기 용량 1, top은 -1로 초기화
- `push`: 포화 상태이면 `capacity *= 2` 하고 `realloc` 수행
- `pop`: 최상단 요소 제거 후 반환
- `free`: 메모리 해제

실행 순서

1. `init_stack(&s)` → 스택 초기화 (용량 1)
2. `push(&s, 1)` → 스택: [1]
3. `push(&s, 2)` → 스택 크기 2로 증가, [1, 2]
4. `push(&s, 3)` → 스택 크기 4로 증가, [1, 2, 3]
5. `pop(&s)` → 출력: 3
6. `pop(&s)` → 출력: 2
7. `pop(&s)` → 출력: 1
8. `free(s.data)` → 메모리 해제

실행 결과

3
2
1

프로그램 4.6 - 괄호 검사 프로그램

코드 전문

```
#include <stdio.h>
#include <stdlib.h>

typedef int element;
typedef struct {
    element *data;
    int capacity;
    int top;
} StackType;

void init_stack(StackType *s) {
    s->top = -1;
    s->capacity = 1;
    s->data = (element *)malloc(s->capacity * sizeof(element));
}

int is_empty(StackType *s) { return (s->top == -1); }
int is_full(StackType *s) { return (s->top == (s->capacity - 1)); }

void push(StackType *s, element item) {
    if (is_full(s)) {
        s->capacity *= 2;
        s->data = (element *)realloc(s->data, s->capacity * sizeof(element));
    }
    s->data[++(s->top)] = item;
}

element pop(StackType *s) {
    if (is_empty(s)) {
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    }
    return s->data[(s->top)--];
}

int check_matching(const char *in) {
    StackType s;
    init_stack(&s);

    for (int i = 0; in[i] != '\0'; i++) {
```

```

    char ch = in[i];
    if (ch == '(' || ch == '[' || ch == '{') {
        push(&s, ch);
    } else if (ch == ')' || ch == ']' || ch == '}') {
        if (is_empty(&s)) {
            free(s.data);
            return 0;
        }
        char open_ch = pop(&s);
        if ((open_ch == '(' && ch != ')') ||
            (open_ch == '[' && ch != ']') ||
            (open_ch == '{' && ch != '}')) {
            free(s.data);
            return 0;
        }
    }
}

int result = is_empty(&s);
free(s.data);
return result;
}

int main() {
    const char *p = "{ A[(i+1)]=0; }";
    if (check_matching(p) == 1)
        printf("%s 괄호검사성공\n", p);
    else
        printf("%s 괄호검사실패\n", p);
    return 0;
}

```

설명

이 프로그램은 **괄호의 짝이 올바르게 맞는지** 검사하는 괄호 유효성 확인기입니다.
 스택을 사용하여 열린 괄호를 저장하고, 닫힌 괄호가 들어올 때 짝이 맞는지 확인합니다.

- **push**: 열린 괄호 저장
- **pop**: 닫힌 괄호가 들어올 때 가장 마지막 괄호와 비교
- 짝이 맞지 않거나 닫힌 괄호만 있을 경우 실패
- 모든 괄호가 짝이 맞고 스택이 비어 있으면 성공

실행 순서

1. 문자열 순회하면서 열린 괄호 → push
2. 닫힌 괄호 → pop하여 짝 비교
3. 끝까지 확인 후 스택이 비어있으면 유효한 괄호

☑ 실행 결과

```
{ A[(i+1)]=0; } 괄호검사성공
```

📌 프로그램 4.7 - 후위표기식 계산

🔍 코드 전문

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STACK_SIZE 100

typedef char element;
typedef struct {
    element *data;
    int capacity;
    int top;
} StackType;

void init_stack(StackType *s) {
    s->top = -1;
    s->capacity = 1;
    s->data = (element *)malloc(s->capacity * sizeof(element));
}

int is_empty(StackType *s) {
    return (s->top == -1);
}

int is_full(StackType *s) {
    return (s->top == (s->capacity - 1));
}

void push(StackType *s, element item) {
    if (is_full(s)) {
        s->capacity *= 2;
        s->data = (element *)realloc(s->data, s->capacity * sizeof(element));
    }
    s->data[++(s->top)] = item;
}

element pop(StackType *s) {
    if (is_empty(s)) {
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    }
}
```

```

        return s->data[(s->top)--];
    }

    int eval(char exp[]) {
        int op1, op2, value, i = 0;
        int len = strlen(exp);
        char ch;
        StackType s;

        init_stack(&s);
        for (i = 0; i < len; i++) {
            ch = exp[i];
            if (ch != '+' && ch != '-' && ch != '*' && ch != '/') {
                value = ch - '0';
                push(&s, value);
            } else {
                op2 = pop(&s);
                op1 = pop(&s);
                switch (ch) {
                    case '+': push(&s, op1 + op2); break;
                    case '-': push(&s, op1 - op2); break;
                    case '*': push(&s, op1 * op2); break;
                    case '/': push(&s, op1 / op2); break;
                }
            }
        }
        return pop(&s);
    }

    int main() {
        int result;
        printf("후위표기식은 82/3-32*+\n");
        result = eval("82/3-32*+");
        printf("결과값은 %d\n", result);
        return 0;
    }

```

설명

이 프로그램은 **후위표기식(Postfix)** 을 계산하는 계산기입니다.

피연산자는 스택에 push하고, 연산자를 만나면 두 개의 피연산자를 pop해서 계산 후 결과를 다시 push합니다.

- 예제 식: $82/3-32*+$
- 연산 우선순위에 따라 괄호 없이 계산 수행 가능

실행 순서

1. 8 → push
2. 2 → push
3. / → $8 / 2 = 4$ → push

4. 3 → push
5. - → 4 - 3 = 1 → push
6. 3 → push
7. 2 → push
8. * → 3 * 2 = 6 → push
9. + → 1 + 6 = 7 → push

☑ 실행 결과

후위표기식은 82/3-32*+
결과값은 7

📌 프로그램 4.8 - 중위 표기식을 후위 표기식으로 변환

🔍 코드 전문

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STACK_SIZE 100

typedef char element;
typedef struct {
    element *data;
    int capacity;
    int top;
} StackType;

void init_stack(StackType *s) {
    s->top = -1;
    s->capacity = 1;
    s->data = (element *)malloc(s->capacity * sizeof(element));
}

int is_empty(StackType *s) {
    return (s->top == -1);
}

int is_full(StackType *s) {
    return (s->top == (s->capacity - 1));
}

void push(StackType *s, element item) {
    if (is_full(s)) {
        s->capacity *= 2;
```

```

        s->data = (element *)realloc(s->data, s->capacity * sizeof(element));
    }
    s->data[++(s->top)] = item;
}

element pop(StackType *s) {
    if (is_empty(s)) {
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    }
    return s->data[(s->top)--];
}

element peek(StackType *s) {
    if (is_empty(s)) {
        fprintf(stderr, "스택 공백 에러 (peek)\n");
        exit(1);
    }
    return s->data[s->top];
}

int prec(char op) {
    switch (op) {
        case '(': case ')': return 0;
        case '+': case '-': return 1;
        case '*': case '/': return 2;
    }
    return -1;
}

void infix_to_postfix(char exp[]) {
    int i;
    char ch, top_op;
    int len = strlen(exp);
    StackType s;

    init_stack(&s);
    for (i = 0; i < len; i++) {
        ch = exp[i];
        switch (ch) {
            case '+': case '-': case '*': case '/':
                while (!is_empty(&s) && (prec(ch) <= prec(peek(&s))))
                    printf("%c", pop(&s));
                push(&s, ch);
                break;
            case '(':
                push(&s, ch);
                break;
            case ')':
                top_op = pop(&s);
                while (top_op != '(') {
                    printf("%c", top_op);
                    top_op = pop(&s);
                }
        }
    }
}

```

```

        break;
    default:
        printf("%c", ch);
        break;
    }
}
while (!is_empty(&s))
    printf("%c", pop(&s));
}

int main() {
    char *s = "(2+3)*4+9";
    printf("중위표시수식: %s\n", s);
    printf("후위표기수식: ");
    infix_to_postfix(s);
    printf("\n");
    return 0;
}

```

☁ 설명

이 프로그램은 중위 표기식(사람이 읽는 수식)을 후위 표기식(컴퓨터가 계산하기 쉬운 형식)으로 변환합니다. 연산자의 우선순위와 괄호의 중첩을 고려하여 변환하며, 스택을 이용해 연산자를 임시 저장합니다.

⚙ 실행 순서 (예시 입력: "(2+3)*4+9")

1. (→ push
2. 2 → 출력
3. + → push
4. 3 → 출력
5.) → + pop → 출력
6. * → push
7. 4 → 출력
8. + → * pop 후 출력, + push
9. 9 → 출력
10. 남은 연산자 + pop → 출력

최종 출력: 23+4*9+

☑ 실행 결과

중위표시수식: (2+3)*4+9
 후위표기수식: 23+4*9+

프로그램 4.9 - 미로 탐색 프로그램

코드 전문

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 6

typedef struct {
    short r;
    short c;
} element;

typedef struct {
    element *data;
    int capacity;
    int top;
} StackType;

void init_stack(StackType *s) {
    s->top = -1;
    s->capacity = 1;
    s->data = (element *)malloc(s->capacity * sizeof(element));
}

int is_empty(StackType *s) {
    return (s->top == -1);
}

int is_full(StackType *s) {
    return (s->top == (s->capacity - 1));
}

void push(StackType *s, element item) {
    if (is_full(s)) {
        s->capacity *= 2;
        s->data = (element *)realloc(s->data, s->capacity * sizeof(element));
    }
    s->data[++(s->top)] = item;
}

element pop(StackType *s) {
    if (is_empty(s)) {
        fprintf(stderr, "스택 공백 에러\n");
        exit(1);
    }
    return s->data[(s->top)--];
}

element peek(StackType *s) {
```



```

    if (is_empty(s)) {
        fprintf(stderr, "스택 공백 에러 (peek)\n");
        exit(1);
    }
    return s->data[s->top];
}

element here = {1, 0}, entry = {1, 0};

char maze[MAX_SIZE][MAX_SIZE] = {
    {'1', '1', '1', '1', '1', '1'},
    {'e', '0', '1', '0', '0', '1'},
    {'1', '0', '0', '0', '1', '1'},
    {'1', '0', '1', '0', '1', '1'},
    {'1', '0', '1', '0', '0', 'x'},
    {'1', '1', '1', '1', '1', '1'},
};

void push_loc(StackType *s, int r, int c) {
    if (r < 0 || c < 0 || r >= MAX_SIZE || c >= MAX_SIZE) return;
    if (maze[r][c] != '1' && maze[r][c] != '.') {
        element tmp = {r, c};
        push(s, tmp);
    }
}

void maze_print(char maze[MAX_SIZE][MAX_SIZE]) {
    printf("\n");
    for (int r = 0; r < MAX_SIZE; r++) {
        for (int c = 0; c < MAX_SIZE; c++) {
            printf("%c", maze[r][c]);
        }
        printf("\n");
    }
}

int main() {
    int r, c;
    StackType s;
    init_stack(&s);
    here = entry;

    while (maze[here.r][here.c] != 'x') {
        r = here.r;
        c = here.c;
        maze[r][c] = '.';
        maze_print(maze);
        push_loc(&s, r - 1, c);
        push_loc(&s, r + 1, c);
        push_loc(&s, r, c - 1);
        push_loc(&s, r, c + 1);

        if (is_empty(&s)) {
            printf("실패\n");
        }
    }
}

```

```

        return 0;
    } else {
        here = pop(&s);
    }
}

maze_print(maze);
printf("성공\n");
return 0;
}

```

설명

이 프로그램은 DFS(깊이 우선 탐색)를 기반으로 한 **스택을 이용한 미로 탐색 알고리즘**입니다.
미로에서 시작점 **e**에서 도착점 **x**까지 경로를 찾고, 경로는 **.**으로 표시됩니다.

- `push_loc()` : 유효한 이동 좌표를 스택에 push
- `maze[r][c] = '.'` : 방문한 경로를 마킹
- 탐색 도중 가능한 모든 방향을 push한 뒤, 다시 pop하여 탐색 계속

실행 순서

1. 시작 위치 **entry**에서 시작
2. 상하좌우 가능한 위치를 스택에 push
3. pop으로 다음 이동 위치 선택
4. 도착점 **x**에 도달 시 종료 및 성공 메시지
5. 스택이 비면 더 이상 경로 없음 → 실패

실행 결과 (예시)

```

e.1...
111...
성공

```

※ 실제 경로에 따라 경유한 지점은 **.**으로 표시되며 매 스텝마다 미로가 출력됩니다.